

BMAD Workflow V1.0 — Visual Guide

AI-Driven Engineering Workflow untuk Production-Ready Apps

2026-05-31



BMAD Workflow V1.0 — Visual Guide

Untuk siapa dokumen ini? Developer yang baru kenal AI-driven workflow, atau yang udah pakai tapi mau lebih terstruktur. Dokumen ini visual-heavy supaya gampang dipahami sambil ngeprint.

Versi: v1.1 — Updated 2026-05-31 dengan codegraph integration + multi-agent compatibility

1. Pengenalan

1.1 Apa Itu BMAD?

BMAD adalah singkatan dari 4 tahap berpikir:

B	=	BRAINSTORM	→	Pahami masalah real
M	=	MODEL	→	Definisikan produk
A	=	ARCHITECT	→	Design sistem teknis
D	=	DEVELOP	→	Build + audit

Setiap tahap dieksekusi dengan AI agent (Claude Code, Cursor, Codex, Gemini, opencode, Hermes Agent, Antigravity, Kiro, dll) lewat **PHASE 0–4** yang konkret, bukan teori.

1.2 Multi-Agent Compatible

Workflow ini **agent-agnostic**. Bisa dipake dengan 8 AI agent populer yang support MCP (Model Context Protocol):

```
flowchart LR
```

```
A[BMAD Workflow] -->|works with| B[Claude Code<br/>Anthropic]
```

```
A -->|works with| C[Cursor<br/>Multi-model]
```

```
A -->|works with| D[Codex CLI<br/>OpenAI]
```

```
A -->|works with| E[opencode<br/>Multi-model]
```

```
A -->|works with| F[Hermes Agent<br/>Multi-model]
```

```
A -->|works with| G[Gemini CLI<br/>Google]
```

```
A -->|works with| H[Antigravity IDE<br/>Multi-model]
```

```
A -->|works with| I[Kiro<br/>Multi-model]
```

```
style A fill:#fff3cd,stroke:#856404,stroke-width:3px
```

1.3 Kenapa Pakai Workflow Ini?

```
graph LR
```

```
A[❌ Tanpa Workflow] -->|hasil| B[Code jalan tapi:<br/>- Security gap<br/>- A11y diabaikan]
```

```
C[✅ Pakai BMAD] -->|hasil| D[Code aman + cepat:<br/>- WCAG AA<br/>- Lighthouse ≥ 90<br/>- Bundle size kecil]
```

```
style A fill:#f8d7da
```

```
style B fill:#f8d7da
```

```
style C fill:#d4edda
```

```
style D fill:#d4edda
```

1.4 Komponen Utama

```
mindmap
  root((BMAD<br/>Workflow))
    Planning
      project-init.md
      PRD template
      SRD template
      SYSTEM_MAP template
    Build
      task-executor.md
      auto-code-review.md
      bug-hunter.md
    Quality Gate
      auto-security-agent.md
      OWASP ZAP DAST
      Lighthouse
      axe-core ally
    CI/CD
      audit-pipeline.yml
      security-scan.yml
      ci-gate.js
      dependabot.yml
    MCP Layer
      lean-ctx
      codegraph
      you-com optional
      github optional
```

2. Architecture Project

2.1 Struktur Direktori (Visual)

```
BMAD-Workflow-V1.0/
├── DOCUMENTATION (read-first)
│   ├── README.md           # Overview cepat
│   └── WORKFLOW_GUIDE.md   # Guide ini
├── ENTRY POINT
│   └── bmad-main.md        # ★ Wajib upload tiap sesi
├── templates/              # Cetakan untuk project lo
│   ├── PRD.template.md
│   ├── SRD.template.md
│   ├── SYSTEM_MAP.template.md # ⚠ Paling penting
│   └── mcp.template.json
├── AGENT FILES (per phase)
│   ├── project-init.md     # PHASE 0: Onboarding
│   ├── task-executor.md    # PHASE 1: Build planning
│   ├── auto-code-review.md # PHASE 2: Code review
│   ├── bug-hunter.md       # PHASE 2: Debug
│   └── auto-security-agent.md # PHASE 3-4: Audit
├── ⚙ CI/CD (copy ke project)
│   ├── .github/
│   │   ├── dependabot.yml
│   │   └── workflows/
│   │       ├── audit-pipeline.yml
│   │       └── security-scan.yml
│   └── scripts/
│       └── ci-gate.js
└── REFERENCE
    ├── audit-checklist.md
    └── accessibility-wcag.md
```

2.2 Hubungan Antar File

```
graph TD
    USER([User])
    AI([AI Agent<br/>Claude/Cursor/Codex/dll])

    USER -->|baca dulu| README[README.md]
    USER -->|baca lengkap| GUIDE[WORKFLOW_GUIDE.md<br/>file ini]

    USER -->|upload setiap sesi| MAIN[bmad-main.md<br/>★ entry point]
    USER -->|copy ke project| TPL[templates/]
    TPL -->|hasil isi| WORK[Working files<br/>PRD, SRD, SYSTEM_MAP, mcp.json]
    USER -->|upload working files| AI

    USER -->|upload sesuai phase| AGENTS[Agent files<br/>project-init, task-executor, dll]
    AGENTS --> AI

    USER -->|copy ke project| CI[.github/ + scripts/]
    CI -->|push ke GitHub| GH[GitHub Actions<br/>auto-validate]

    MCP[MCP Servers<br/>lean-ctx + codegraph] -.->|enhance| AI

    AI -->|generate output| USER
    GH -->|pass/fail| USER

    style MAIN fill:#fff3cd,stroke:#856404,stroke-width:3px
    style AI fill:#cfe2ff
    style USER fill:#d4edda
    style MCP fill:#e2d9f3
```

3. Mental Model: 4 Tipe File

Sebelum mulai, lo HARUS paham 4 tipe file di repo ini.

3.1 Diagram Alur File

```
flowchart LR
    subgraph TOOLKIT["📦 Repo BMAD-Workflow (toolkit)"]
        D[📄 Docs]
        T[📁 Templates]
        A[📁 Agent Files]
        C[⚙️ CI/CD]
    end

    subgraph PROJECT["🔗 Project Lo"]
        W[📄 Working Files<br/>PRD, SRD, SYSTEM_MAP]
        CP[⚙️ CI/CD copy]
    end

    subgraph CLAUDE["☁️ Claude.ai"]
        AI[🧠 AI Agent]
    end

    subgraph GITHUB["👤 GitHub"]
        GA[📁 Actions Runner]
    end

    D -->|baca dulu| USER([User])
    T -->|copy + isi| W
    A -->|upload sesi| AI
    C -->|copy| CP
    W -->|upload| AI
    CP -->|push trigger| GA

    AI -->|output| USER
    GA -->|verdict| USER

    style TOOLKIT fill:#e2d9f3
    style PROJECT fill:#d4edda
    style CLAUDE fill:#cfe2ff
    style GITHUB fill:#fff3cd
```

3.2 Tabel Perbedaan

Tipe	Ada di Mana	Diisi Siapa	Di-update Kapan
 Docs	Repo workflow	Maintainer toolkit	Kalau ada update workflow
 Templates	/templates/	Maintainer toolkit	Update terstruktur
 Working Files	Project lo	Lo + AI	Setiap progress project
 Agent Files	Repo workflow	Maintainer toolkit	Stabil, jarang berubah
 CI/CD	Repo workflow + project lo	Lo customize	Project-specific tweaks

3.3 Aturan Emas

```
Repo BMAD-Workflow = TOOLKIT (cetakan)
Project lo          = TEMPAT KERJA

❌ Jangan isi PRD.md di repo workflow
✅ Copy template dulu, baru isi di project
```


4. Pipeline 7 Phase Detail

4.1 Big Picture Pipeline

```
flowchart TB
    START([💡 Project Idea])

    subgraph PLAN["🗨️ PLANNING (PHASE 0 - 0.6)"]
        P0[PHASE 0<br/>project-init<br/>6 questions]
        P05[PHASE 0.5<br/>visual insight<br/>wireframe]
        P06[PHASE 0.6<br/>SYSTEM_MAP<br/>architecture]
    end

    subgraph BUILD["🏗️ BUILD (PHASE 1 - 2)"]
        P1[PHASE 1<br/>task-executor<br/>breakdown]
        P2[PHASE 2<br/>build + iterate<br/>review/debug loop]
    end

    subgraph AUDIT["✅ AUDIT (PHASE 3)"]
        P3A[3a. SAST<br/>+ deps]
        P3B[3b. DAST<br/>OWASP ZAP]
        P3C[3c. WCAG<br/>axe-core]
        P3D[3d. Performance<br/>Lighthouse]
    end

    subgraph GATE["🚦 GATE (PHASE 4)"]
        P4[CI Gate<br/>aggregate verdict]
    end

    DEPLOY([🚀 Production])
    FAIL([❌ Back to Build])

    START --> P0 --> P05 --> P06
    P06 --> P1 --> P2 --> P3A
    P3A --> P3B --> P3C --> P3D --> P4
    P4 -->|✅ Pass| DEPLOY
    P4 -->|❌ Fail| FAIL
    FAIL -. ->|fix issues| P2

    style START fill:#cfe2ff
    style DEPLOY fill:#d4edda
    style FAIL fill:#f8d7da
```

4.2 PHASE 0 — Project Init

Tujuan: Setup project dari nol — generate PRD + SRD.

Cara pakai:

```
Step 1: Upload bmad-main.md + project-init.md ke Claude.ai
Step 2: Ketik: "Jalankan PHASE 0: project-init"
Step 3: Jawab 6 pertanyaan:
    1. Nama project?
    2. Masalah yang diselesaikan?
    3. Target user?
    4. 3-5 fitur utama?
    5. Tech stack (FE/BE/DB)?
    6. Deploy di mana?
Step 4: Claude generate PRD.md + SRD.md
Step 5: Copy-paste output ke working files lo
```

Hasil:

-  PRD.md terisi (product requirements)
-  SRD.md terisi (system requirements)

4.3 PHASE 0.5 — Visual Insight

Tujuan: Alignment visual sebelum nulis kode.

Cara pakai:

```
Step 1: Upload bmad-main.md + PRD.md (yang udah jadi)
Step 2: Ketik: "Jalankan PHASE 0.5: visual insight"
Step 3: Claude generate:
    - Diagram arsitektur high-level
    - User flow utama
    - Data model / ERD (kalau relevan)
    - Wireframe kasar (kalau UI-heavy)
```

Output yang lo dapet: Mermaid diagrams + ASCII wireframes.

4.4 PHASE 0.6 — Isi SYSTEM_MAP.md

Tujuan: Definisikan arsitektur lengkap supaya AI nggak nebak-nebak.

flowchart LR

A[PRD + SRD] --> B[Template
SYSTEM_MAP.template.md]

B --> C[Isi: tech stack,
entrypoints, flows,
boundaries, deps]

C --> D[SYSTEM_MAP.md
siap dipakai]

D --> E[Upload ke Claude
setiap sesi]

style D fill:#d4edda,stroke:#155724,stroke-width:3px

Section yang harus diisi:

- Project Snapshot (nama, type, goal, stage)
- Tech Stack (FE/BE/DB/cache/hosting)
- Top-Level Layout (folder structure)
- Entry Points (route, API, worker, CLI)
- Runtime Flows (auth, business, payment, notification)
- Module Boundaries
- Integration Points
- Data Model

 **PENTING:** SYSTEM_MAP.md adalah file paling kritis. AI baca ini untuk paham project lo. Kalau kosong, AI akan nebak dan hasilnya jelek.

4.5 PHASE 1 — Task Executor

Tujuan: Convert PRD + SRD jadi task list konkret.

Input: PRD + SRD + SYSTEM_MAP **Output:**

```
Feature: User Authentication
├─ FE Tasks:
│   ├─ Login form component
│   ├─ Register form component
│   └─ Session management hook
├─ BE Tasks:
│   ├─ Auth middleware
│   ├─ JWT generator
│   └─ Password hasher (bcrypt)
├─ API Endpoints:
│   ├─ POST /api/auth/login
│   ├─ POST /api/auth/register
│   └─ POST /api/auth/refresh
└─ DB Schema:
    └─ users (id, email, password, role, ...)
```

Implementation Order:

1. DB schema + migration
2. BE: auth service + JWT
3. BE: API endpoints + middleware
4. FE: forms + state mgmt
5. FE: integration + error handling

4.6 PHASE 2 — Build + Iterate

Tujuan: Loop build → review → debug.

```
stateDiagram-v2
    [*] --> Build
    Build --> Review : kode ditulis
    Review --> Debug : ada issue
    Review --> Test : kode bersih
    Debug --> Build : fix applied
    Test --> NextFeature : pass
    Test --> Build : fail
    NextFeature --> Build : ada feature lain
    NextFeature --> [*] : semua done

    note right of Review
        Pakai auto-code-review.md
    end note

    note right of Debug
        Pakai bug-hunter.md
    end note
```

Sebelum edit file apapun, cek SYSTEM_MAP.md dulu:

```
Target file      : src/api/auth/login.ts
Entrypoint      : POST /api/auth/login
Upstream callers : LoginForm.tsx
Downstream deps  : authService, userRepo
Risk            : Medium (auth-critical)
```

4.7 PHASE 3 — Audit Pipeline

Tujuan: Validasi quality sebelum deploy.

```
flowchart TB
    START([App ready for audit])
    
    subgraph SAST["🔒 PHASE 3a: SAST"]
        S1[npm audit]
        S2[Snyk]
        S3[CodeQL]
        S4[gitleaks]
    end
    
    subgraph DAST["🚧 PHASE 3b: DAST"]
        D1["OWASP ZAP<br/>baseline scan"]
        D2["Trivy<br/>container scan"]
    end
    
    subgraph A11Y["🗎 PHASE 3c: WCAG"]
        W1[axe-core CLI]
        W2[Manual checklist]
    end
    
    subgraph PERF["⚡ PHASE 3d: Performance"]
        P1[Lighthouse]
    end
    
    REPORT[📄 Generate reports]

    START --> SAST --> DAST --> A11Y --> PERF --> REPORT

    style SAST fill:#fff3cd
    style DAST fill:#f8d7da
    style A11Y fill:#cfe2ff
    style PERF fill:#d4edda
```

Hasil: 4 report files

- `npm-audit.json` + `snyk.sarif`
- `zap-report.json`

- `axe-results.json`
- `lighthouse-report.json`

4.8 PHASE 4 — CI Gate

Tujuan: Aggregate semua audit → satu verdict PASS / FAIL.

```
flowchart LR
    A[zap-report.json] --> CI[ci-gate.js]
    B[lighthouse-report.json] --> CI
    C[axe-results.json] --> CI

    CI --> D{All checks<br/>pass?}
    D -->|All ≥ thresholds| PASS[✅ PASS<br/>exit 0]
    D -->|Any below| FAIL[❌ FAIL<br/>exit 1]
    D -->|No data| NODATA[⚠️ FAIL<br/>exit 2]

    PASS --> DEPLOY[🚀 Deploy]
    FAIL --> FIX[🔧 Back to PHASE 2]
    NODATA --> RERUN[🔄 Rerun audit]

    style PASS fill:#d4edda
    style FAIL fill:#f8d7da
    style NODATA fill:#fff3cd
```

Thresholds:

Check	Threshold	Block?
Lighthouse Performance	≥ 90	✅
Lighthouse Accessibility	≥ 90	✅
Lighthouse Best Practices	≥ 90	✅
Lighthouse SEO	≥ 90	✅
DAST High alerts	0	✅
WCAG Critical	0	✅
WCAG Serious	0	✅

5. Security Architecture

5.1 Defense in Depth

```
flowchart TB
    CODE[Source Code]

    subgraph L1["LAYER 1: Static Analysis"]
        L1A[npm audit<br/>built-in CVE]
        L1B[Snyk<br/>deeper + license]
        L1C[CodeQL<br/>code patterns]
    end

    subgraph L2["LAYER 2: Secret Scanning"]
        L2A[gitleaks<br/>code + git history]
    end

    subgraph L3["LAYER 3: Dynamic Analysis"]
        L3A[OWASP ZAP<br/>runtime DAST]
    end

    subgraph L4["LAYER 4: Container & Infra"]
        L4A[Trivy<br/>container scan]
    end

    subgraph L5["LAYER 5: Continuous"]
        L5A[Dependabot<br/>weekly auto-PR]
    end

    CODE --> L1 --> L2 --> L3 --> L4
    L5 -.->|continuous| CODE

    DEPLOY[🚀 Production]
    L4 --> DEPLOY

    style L1 fill:#cfe2ff
    style L2 fill:#fff3cd
    style L3 fill:#f8d7da
    style L4 fill:#e2d9f3
    style L5 fill:#d4edda
```

5.2 OWASP Top 10 Coverage

Risk	Detection Layer
A01 Broken Access Control	DAST + manual SAST checklist
A02 Cryptographic Failures	SAST + manual review
A03 Injection	DAST + CodeQL
A04 Insecure Design	Manual (PHASE 0–0.6)
A05 Security Misconfiguration	DAST + Trivy
A06 Vulnerable Components	npm audit + Snyk + Dependabot
A07 Auth Failures	DAST + manual SAST
A08 Software & Data Integrity	Trivy + SBOM (optional)
A09 Logging & Monitoring	Manual checklist
A10 SSRF	DAST + CodeQL

6. Workflow End-to-End — Real Example

Scenario: Bikin "Todo App with Auth"

```
sequenceDiagram
    actor Dev as  Developer
    participant Local as  Local Machine
    participant Claude as  Claude.ai
    participant Git as  GitHub
    participant CI as  CI Pipeline

    Note over Dev,CI: PHASE 0 - Setup
    Dev->>Local: Copy templates ke project
    Dev->>Claude: Upload bmad-main + project-init
    Dev->>Claude: "PHASE 0: project-init"
    Claude->>Dev: Tanya 6 pertanyaan
    Dev->>Claude: Jawab (Todo app, individuals, ...)
    Claude->>Dev: PRD.md + SRD.md
    Dev->>Local: Save PRD + SRD

    Note over Dev,CI: PHASE 0.5-0.6 - Architecture
    Dev->>Claude: "PHASE 0.5: visual"
    Claude->>Dev: Wireframe + flow diagram
    Dev->>Local: Isi SYSTEM_MAP.md

    Note over Dev,CI: PHASE 1 - Task Plan
    Dev->>Claude: "PHASE 1: task-executor"
    Claude->>Dev: Task breakdown per fitur

    Note over Dev,CI: PHASE 2 - Build
    loop Per Feature
        Dev->>Local: Code feature
        Dev->>Claude: "review kode ini"
        Claude->>Dev: Issues + fixes
        Dev->>Local: Apply fixes
    end

    Note over Dev,CI: PHASE 3-4 - Audit + Deploy
    Dev->>Git: git push
    Git->>CI: Trigger workflow
    CI->>CI: SAST + DAST + Lighthouse + allly
    CI->>CI: ci-gate.js aggregate
    alt All Pass
        CI->>Dev:  Green check
        Dev->>Git: Merge to main
    else Any Fail
        CI->>Dev:  Block + report
        Dev->>Local: Fix issues
        Dev->>Git: Push again
    end
end
```

7. Cheat Sheet

7.1 Trigger Commands

Phase	Command di Claude
0	Jalankan PHASE 0: project-init
0.5	Jalankan PHASE 0.5: visual insight
0.6	Bantu isi SYSTEM_MAP.md berdasarkan PRD + SRD ini: [paste]
1	Jalankan PHASE 1: task-executor
2 review	Jalankan PHASE 2: review kode ini [paste]
2 debug	Jalankan PHASE 2: fix this error [paste error + code]
3	Jalankan PHASE 3: full audit
4	Jalankan PHASE 4: validasi hasil Lighthouse ini

7.2 File yang Wajib Diupload

SETIAP SESI:

- ✓ bmad-main.md
- ✓ SYSTEM_MAP.md (kalau udah ada)

PHASE 0: + project-init.md

PHASE 1-2: + PRD.md + SRD.md + task-executor.md

PHASE 2 review: + auto-code-review.md

PHASE 2 debug: + bug-hunter.md

PHASE 3-4: + auto-security-agent.md + accessibility-wcag.md + audit-checklist.md

7.3 Common Pitfalls

❌ Salah	✅ Benar
Edit PRD.md di repo workflow	Copy template dulu, edit di project lo
Skip SYSTEM_MAP.md	Isi dulu — paling kritis
Upload semua file tiap sesi	Upload per phase, hemat token
Ignore audit fail	Fix dulu, jangan skip
Pakai <code>npm install</code> di CI	Pakai <code>npm ci</code> (deterministic)
Image <code>owasp/zap2docker-stable</code>	Pakai <code>ghcr.io/zaproxy/zaproxy:stable</code>

8. Convert Guide ini ke PDF

Opsi A: Built-in script 🌟 (recommended)

Script `scripts/build-pdf.js` udah include — render Mermaid native, pakai Chrome/Edge yang udah ada di sistem (no extra install, cuma butuh `node`).

```
node scripts/build-pdf.js
# Output: WORKFLOW_GUIDE.pdf (~1.7 MB dengan semua Mermaid diagram)
```

Custom input/output:

```
node scripts/build-pdf.js docs/CUSTOM.md out/custom.pdf
```

Yang di-handle otomatis:

- ✅ Mermaid diagrams (render via CDN, headless Chrome)
- ✅ GFM tables, code blocks, blockquote
- ✅ Cover page dari YAML frontmatter
- ✅ Auto-install `marked` dependency (one-time, ~50 KB)
- ✅ Cross-platform (Windows/Mac/Linux — auto-detect Chrome/Edge)

Opsi B: Pandoc (CLI)

```
# Install pandoc dulu (sekali aja)
# Windows: https://pandoc.org/installing.html

pandoc WORKFLOW_GUIDE.md \
  -o WORKFLOW_GUIDE.pdf \
  --pdf-engine=xelatex \
  --toc \
  --toc-depth=3 \
  -V geometry:margin=2cm \
  -V mainfont="Calibri"
```

⚠ Pandoc nggak render Mermaid native. Pre-render diagram ke `.png` pakai `mermaid-cli`, atau pakai Opsi A.

Opsi C: VSCode Extension

1. Install extension `Markdown PDF` (yzane.markdown-pdf)
2. Buka `WORKFLOW_GUIDE.md`
3. Right-click → `Markdown PDF: Export (pdf)`

Opsi D: Typora

1. Buka `WORKFLOW_GUIDE.md` di Typora
2. File → Export → PDF

9. Resources & Next Steps

Dokumentasi Tambahan

-  [README.md](#) — Overview cepat
-  [bmad-main.md](#) — Entry point + philosophy
-  [auto-security-agent.md](#) — Security pipeline detail
-  [accessibility-wcag.md](#) — WCAG guidelines
-  [audit-checklist.md](#) — Manual checklist

External Tools

- [OWASP ZAP](#) — DAST scanner

- [Lighthouse](#) — Performance audit
- [axe-core](#) — Accessibility testing
- [Snyk](#) — Dependency security
- [Dependabot](#) — Auto updates
- [CodeQL](#) — SAST
- [gitleaks](#) — Secret scanner
- [Trivy](#) — Container scanner

Standards & References

- [WCAG 2.1 Quick Reference](#)
- [OWASP Top 10](#)
- [Web Content Accessibility Guidelines](#)
- [Mozilla Web Security Guidelines](#)

10. Closing

Quality bukan optional – quality itu GATED.

✅ Lolos pipeline = layak deploy

❌ Gagal = balik ke PHASE 2

Trust the process. The pipeline catches what humans miss.

Pertanyaan? Buka issue di repo, atau diskusi di community channel.

Lisensi: MIT — silakan customize sesuai kebutuhan project lo.

Dokumen ini di-generate sebagai bagian dari BMAD Workflow V1.0. Last updated: 2026-05-31.